

GONES45

Mémoire technique — état au 5 août 2026

Élément	Valeur
Production	gones45140.github.io/gones45 (branche principal)
Dépôt de test	gones45140/fenotte45 (chantier Supabase)
Architecture	app.js (29 184 lignes) + index.html + sw.js
Proxy	fd-proxy.touraine-antoine.workers.dev (Cloudflare Worker + D1)
Données synchronisées	données/paris_antoine.json (chiffré), joueurs.json, stats.json
Contrainte absolue	100 % gratuit — aucune offre payante

Ce document couvre l'infrastructure, les procédures d'urgence, les pièges vérifiés en production et les incidents résolus. Il ne reprend pas l'historique des fonctionnalités : il est fait pour être ouvert un jour de panne.

1. Procédures d'urgence

À lire en premier. Deux pertes de données sont survenues en une semaine (30 juillet et 4 août) — dans les deux cas, une manipulation faite dans le mauvais ordre a aggravé la situation.

Règle numéro un : sauvegarder avant de toucher

Aucune manipulation de synchronisation ne se fait sans avoir sorti l'état du navigateur au préalable. Dans la console de la page en production :

```
const b = new Blob([localStorage.getItem('g45v5')], {type:'application/json'});
const a = document.createElement('a');
a.href = URL.createObjectURL(b);
a.download = 'g45v5-' + Date.now() + '.json';
a.click();
```

Ne jamais forcer un envoi quand le local est vide

_g45PushBetsGithub(true) écrase la copie distante. Si le local a perdu des données, cette commande détruit la seule sauvegarde intacte. En cas de perte apparente, on ne pousse RIEN tant que l'origine n'est pas comprise.

Où retrouver les données

- **GitHub** — données/paris_antoine.json, onglet History. Chaque commit est un instantané complet et rien n'est jamais perdu, même après un écrasement. Ouvrir le commit voulu, bouton Raw.
- **Le téléphone** — il n'a jamais ouvert fenotte45, donc jamais subi les effets de bord. Le 4 août, il portait l'état complet (16 781,04 €) pendant que le PC n'affichait que 265 €. C'est souvent la copie de référence.
- **Dropbox** — historique des versions conservé 30 jours, clic droit sur le fichier, Historique des versions.

Chiffrement des paris : le piège à connaître

Les paris sont chiffrés dans paris_antoine.json. La clé de déchiffrement est, par défaut, **le token GitHub lui-même** :

```
function _g45BetsPass(){ return localStorage.getItem('g45_bets_pass') ||
localStorage.getItem('gones45_github_token'); }
```

Régénérer le token GitHub sans précaution rend les paris illisibles. Avant toute rotation, découpler la clé du token, sur CHAQUE appareil :

```
localStorage.setItem('g45_bets_pass', localStorage.getItem('gones45_github_token'));
```

Sur mobile, passer par Réglages puis le bouton de mot de passe de chiffrement (g45SetBetsPass) et saisir exactement la même chaîne. Ensuite seulement, le token peut être révoqué et remplacé.

Diagnostic express de la synchronisation

```
console.log('token :', localStorage.getItem('gones45_github_token'));
console.log('betsync :', localStorage.getItem('g45_github_betsync'));
console.log('rangé :', localStorage.getItem('gones45_github_token__off'));
console.log('passe :', localStorage.getItem('g45_bets_pass'));
```

La synchronisation est active si le token existe ET que g45_github_betsync vaut autre chose que '0'. Un betsynchronisation à '0' avec un token présent signifie qu'elle a été éteinte, pas cassée : la remettre à '1' suffit.

Quota de stockage saturé

Cause réelle de la perte du 30 juillet : localStorage plein, save() levait QuotaExceededError, donc g45v5 n'était jamais écrit — l'ajout apparaissait à l'écran puis disparaissait au rechargement. Les caches ajoutés au fil des mois (profils cyclistes, calendriers, cotes, classements) avaient rempli le quota. _g45CachePoids() affiche la

répartition, _g45FreeSpace() purge les caches reconstructibles.

Leçon retenue : quatre correctifs successifs sur la synchronisation GitHub ont échoué parce que le diagnostic était faux. C'est un traceur posé sur localStorage.setItem qui a révélé l'erreur en une manipulation.
Instrumenter avant de corriger.

2. Pièges vérifiés en production

2.1 — app.js contient une zone dupliquée

Lignes 398 à 12 000 environ : toute modification doit être appliquée aux DEUX occurrences. Les deux copies contiennent des fonctions vivantes, elles ne doivent pas être fusionnées.

Audit du 29 juillet : sur 230 fonctions homonymes, 229 étaient identiques au caractère près. Une seule avait divergé — `swlInner`, dont la copie active avait perdu la ligne `window._currentInnerTab = id;`, que lit `smartRefresh`. Résultat : le bouton de rafraîchissement croyait toujours être sur l'onglet Bilan et ne vidait jamais le cache des saisons. Refaire cet audit après toute grosse session : il détecte en quelques secondes une correction appliquée à une seule copie.

2.2 — Les deux dépôts partagent le même localStorage

`gones45140.github.io/gones45/` et `gones45140.github.io/fenotte45/` sont sur la MÊME ORIGINE. Le `localStorage` est cloisonné par origine, jamais par chemin. Tout ce que `fenotte` écrit, supprime ou lit affecte la production.

Conséquences constatées le 4 août :

- La bankroll de production s'affichait dans `fenotte` sur un compte Supabase neuf.
- Un compte Supabase non vide aurait écrasé la production.
- La suppression des tokens côté `fenotte` les a supprimés pour la production, qui s'est retrouvée sans synchronisation GitHub pendant des heures.

Correctif appliqué dans `auth-guard.js` : remappage de `g45v5` vers `g45v5__fen` en interceptant `getItem`, `setItem` et `removeItem` avant l'injection d'`app.js`, et masquage des tokens en lecture au lieu de les supprimer.

2.3 — L'initialisation d'app.js repose sur window.onload

`app.js` pose son initialisation principale sur `window.onload` : 1073 lignes, 46 fonctions déclarées dans cette closure, 16 exports vers `window` qui sont eux-mêmes à l'intérieur, le préremplissage des clés API, les onclick de plusieurs boutons. Tout chargement dynamique d'`app.js` après le passage de l'événement `load` rend ce bloc muet. La parade tient en une ligne : `window.dispatchEvent(new Event('load'))`. Rebrancher les boutons un par un ne marche pas, leurs fonctions n'étant pas globales.

2.4 — Handlers inline et portée globale

Un onclick écrit dans `index.html` résout ses identifiants dans la portée GLOBALE, jamais dans une closure. Cinq fonctions étaient appelées ainsi sans être exportées : `setArchFilter`, `setCardStyle`, `updateCardIntensity`, `loadCalendrier`, `initComparateur`. Symptôme trompeur pour les deux dernières : `showTab` étant global, l'onglet s'ouvrait bien mais restait vide, le second appel de la séquence plantant en silence. Corrigé le 5 août par cinq exports ajoutés en fin de bloc `window.onload`.

2.5 — Divers, vérifiés

- **Deux balises CSP** dans une page : le navigateur applique l'INTERSECTION. Ajouter une politique permissive ne débloque jamais une politique stricte.
- **ESPN indexe une saison par son année de début** — `season=2025` désigne 2025-26. Sans paramètre `season`, l'API sert la saison à venir, donc vide avant sa reprise.
- **Le calendrier d'équipe ESPN renvoie tous les matchs**, joués ou non, avec un simple drapeau `completed`. Sans filtre, un match à venir compte comme un 0-0.
- **Une page de blocage et une absence de CORS** produisent exactement le même message en console. Toujours vérifier le statut HTTP réel avant de conclure.

3. Incident ESPN du 5 août 2026

Symptôme

Fiches Palmeiras et Miami CF bloquées sur « Chargement des 2 dernières saisons... », indéfiniment. En réalité, 36 appels ESPN de l'application étaient concernés.

Diagnostic

Akamai, le CDN d'ESPN, renvoie 403 aux adresses IP de sortie des datacenters. Le pool d'IP des Workers Cloudflare est massivement partagé et classé comme trafic automatisé. La page renvoyée est du HTML, pas du JSON : `r.json()` échoue, le catch avale l'erreur, et l'application se rabat silencieusement sur football-data — qui ne couvre ni le Brésil ni la MLS dans l'offre gratuite.

Ce qui a été essayé, et le résultat

Piste	Résultat
Vrai User-Agent de navigateur + Referer + Sec-Fetch-*	ÉCHEC — le blocage porte sur l'IP, pas sur la signature du client
Relais déployé sur Deno Deploy	ÉCHEC — 403 également, même nature d'IP
sports.core.api.espn.com à la place de site.api	INADAPTÉ — API à \$ref, une requête par entité
football-data avec filtre &competitions=BSA	ÉCHEC — 403 « restricted », le Brésil est hors palier gratuit
Appel direct depuis le navigateur	SUCCÈS — 200, l'IP résidentielle passe

Correctif retenu

Un shim posé en ligne 1 d'app.js — donc hors de la zone dupliquée — réécrit les URL qui passaient par le proxy vers un appel direct. Les deux hôtes ESPN étaient déjà autorisés dans le connect-src du CSP, aucune autre modification n'a été nécessaire.

```
(function(){
var _f = window.fetch;
window.fetch = function(u, o){
try{
if (typeof u === 'string' && u.indexOf('host=espn') >= 0 && u.indexOf('workers.dev') >= 0) {
var p = new URLSearchParams(u.slice(u.indexOf('?') + 1));
var path = p.get('path');
if (path) u = (p.get('host') === 'espnweb'
? 'https://site.web.api.espn.com'
: 'https://site.api.espn.com') + path;
}
}catch(e){}
return _f.call(this, u, o);
};
})();
```

Ce qui reste cassé

Les notifications push. Le cron du Worker interroge ESPN depuis le serveur, donc reste sous blocage Akamai : plus d'alerte composition, coup d'envoi, but ni fin de match, ni sur les favoris, ni sur les paris, ni sur les matchs suivis. Piste envisagée : faire remonter les données par GitHub Actions, dont les runners sont sur d'autres plages d'IP, et servir le JSON depuis raw.githubusercontent.com — déjà autorisé dans le CSP et ouvert en CORS. Non testé à ce jour.

Sonde de contrôle

Le Worker expose une sonde à ouvrir dans un onglet ; elle dira le jour où Akamai relâche la pression :

<https://fd-proxy.touraine-antoine.workers.dev/espn-test>

4. État des sources de données

Source	Usage	État
site.api.espn.com	Calendriers, scores, cotes 1N2, compos, live	OK depuis le navigateur — BLOQUÉ depuis un serveur
sports.core.api.espn.com	Stats joueurs, buteurs, plays rugby	OK
football-data.org	Coupes, mi-temps, championnats européens	OK sur ~12 compétitions ; Brésil et MLS exclus
The Odds API	Cotes over/under et BTTS (via /odds du Worker)	OK — quota 500/mois, cache D1 de 15 min
api-sports	Football, NBA, basket, rugby, F1	OK — porte de sortie principale
api-web.nhle.com	NHL	OK
statsapi.mlb.com	MLB	OK
Jolpica + OpenF1	Résultats et live F1	OK
racecenter (ASO)	Cyclisme	OK
Sofascore (RapidAPI)	Tennis, H2H, effectifs	OK
Groq / Gemini / Mistral	Analyses IA en cascade	OK
thesportsdb.com	Non utilisé	Piste — gratuit, CORS ouvert, couvre BSA et MLS

Le Worker n'est pas un simple relais

Il expose aussi /psub, /ptest, /punsub et /pevents pour les abonnements web-push, avec stockage D1 et signature VAPID, plus /odds, /odds-quota, /vid et un cron. Toute idée de migration hors de Cloudflare doit tenir compte de cette partie, qui a besoin d'un stockage persistant. La bonne approche serait de garder le Worker pour le push et de monter un second proxy ailleurs pour les données.

5. Chantiers ouverts

Sujet	État
Notifications push	Cassées — piste GitHub Actions, non testée
fdFetch : 403 traité comme 429	À corriger — 24 s d'attente inutile par fiche équipe (l. 11789)
Rotation de la clé football-data	À faire — la clé circule en clair dans les URL
Rotation du token GitHub	À faire après découplage de g45_bets_pass sur les deux appareils
fenotte45 / Supabase	Abonnement payant ABANDONNÉ. Reste à décider : version gratuite pour Bruno et JP, ou arrêt
Upsert Supabase sans onConflict	À vérifier si fenotte continue — user_id doit être clé primaire ou unique
Prompt IA tennis	À corriger — format en 5 sets pour les Grands Chelems masculins
Module Tendances	À observer sur une vraie journée de championnat, le 21 août

Document généré le 5 août 2026. Les procédures d'urgence de la section 1 sont celles qui ont manqué lors des deux pertes de données de la semaine — c'est la partie à relire avant toute manipulation de synchronisation, pas après.